# Aula 37: Performance e Otimização

Felipe Augusto Rocha da Silva

Desenvolvimento Front End em React

Programa Capacita

INSTITUTO FEDERAL
Minas Gerais

# Índice


- INTRODUÇÃO
- O CICLO DE RENDERIZAÇÃO
- MEMOIZATION
- LAZY LOADING E SUSPENSE
- ANÁLISE DE PERFORMANCE
- REACT PROFILER
- BOAS PRÁTICAS
- EXERCÍCIOS

# Introdução

POR QUE OTIMIZAR

- Retenção:
  - Aplicações lentas impactam experiência do usuário
  - Usuários abandonam sites lentos
  - Cada segundo de atraso reduz a conversão e a satisfação do cliente

# Introdução

POR QUE OTIMIZAR

- Eficiência:
  - Performance é requisito funcional
  - Menos processamento economiza bateria em dispositivos móveis e reduz o consumo de dados

# Introdução

POR QUE OTIMIZAR

- SEO:
  - O Google prioriza sites com bons indicadores de performance, como o LCP e o CLS (Core Web Vitals)

Otimizar cedo demais pode gerar código complexo. Como regra, primeiro faça funcionar, depois otimize com dados.

# Introdução

PROBLEMAS DE PERFORMANCE

- Re-renderizações excessivas
- Componentes pesados
- Listas grandes
- Bundle JavaScript muito grande

# Introdução

PROBLEMAS DE PERFORMANCE

- Re-renderizações excessivas
- Componentes pesados
- Listas grandes
- Bundle JavaScript muito grande

# O ciclo de renderização

01

## TRIGGER

O gatilho que inicia tudo. Acontece quando há uma mudança de estado (useState) ou quando as props do componente mudam.

02

## RENDER

O React chama seus componentes para calcular o que mudou. É aqui que o Virtual DOM é gerado e comparado (diffing).

03

## COMMIT

A fase final onde o React aplica as mudanças calculadas no DOM real do navegador de forma eficiente.

- Otimiza-se a fase de Render para evitar cálculos desnecessários.

# O ciclo de renderização

COMO O REACT RENDERIZA

- Renderização baseada em estado e props
- Virtual DOM
- Processo de reconciliação (diff)
- Re-renderização
  - Todo componente pai re-renderiza seus filhos
  - Mesmo sem mudança visual

# Memoization

O QUE É

- É um Cache de resultados
- Evita cálculos e renderizações desnecessárias

FORMAS DE USO

- React.memo
- useMemo
- useCallback

# Memoization

React.memo

- Higher Order Component (HOC) que memoriza o resultado da renderização de um componente funcional
- Usado em componentes que renderizam frequentemente com as mesmas props
- Cuidado
  - A comparação de props não é gratuita
  - Evite usar em componentes simples onde custo de comparação supera o ganho

# Memoization

React.memo

- Evita re-render se props não mudarem
- Útil para componentes puros

```
const MyComponent = React.memo(({ data }) => {
  // Só re-renderiza se 'data' mudar
  return (
    <div>
      <h1>{data.title}</h1>
      <p>{data.content}</p>
    </div>
  );
});

export default MyComponent;
```

# Memoization

useMemo

- Hook que permite memorizar o resultado de um cálculo entre renderizações, evitando que ele seja refeito desnecessariamente
- **Objetivo**: Otimizar a performance evitando cálculos pesados em cada render
- **Dependências**: Reexecuta só se algum valor do array mudar
- **Uso Consciente**: Use apenas para operações computacionalmente caras.

# Memoization

useMemo

- Memoriza o valor calculado
- Use apenas para cálculos custosos

```
const expensiveValue = useMemo(() => {
  // Este cálculo só roda se 'data' mudar
  return performHeavyCalculation(data);
}, [data]);


return (
  <div>
    Resultado: {expensiveValue}
  </div>
);
```

# Memoization

useCallback

- O useCallback memoriza a definição de uma função entre renderizações
- Em JavaScript, funções são objetos.
- Sem o hook, uma nova função é criada em cada render, mudando sua referência
- Importante ao passar funções para filhos com React.memo

# Memoization

useCallback

- Memoriza funções
- Evita recriação em cada render
- Importante ao passar callbacks para filhos

```
const handleClick = useCallback(() => {
  console.log('Ação executada!');
}, []); // dependências

// Filho não re-renderiza pois a referência é a mesma
<BotoesOtimizados onClick={handleClick} />
```

# Memoization

BOAS PRÁTICAS

- Não usar em tudo
- Medir antes e depois
- Código mais simples é mais fácil de manter

# Memoization

EXERCÍCIO

- Dado um componente com:
  - Contador
  - Lista grande renderizada
- Tarefa:
  - Identificar re-renderizações desnecessárias
  - Aplicar React.memo, useMemo ou useCallback
  - Comparar comportamento antes e depois

# Lazy Loading e Suspense

PROBLEMA

- Bundle grande
- Tempo alto de carregamento inicial

SOLUÇÃO: CODE SPLITTING

- Carregar código sob demanda
- Não carrega tudo de uma vez
- O Lazy Loading permite dividir sua aplicação em pedaços menores (chunks)

# Lazy Loading e Suspense

CODE SPLITTING

- **React.lazy**: Carrega componentes dinamicamente apenas quando necessário
- **Suspense**: Define um estado de carregamento (fallback) enquanto o componente é baixado
- **Benefício**: Redução drástica no tamanho do bundle inicial e carregamento mais rápido

# Lazy Loading e Suspense

CODE SPLITTING

- **React.lazy**: Carrega componentes dinamicamente apenas quando necessário
- **Suspense**: Define um estado de carregamento (fallback) enquanto o componente é baixado
- **Benefício**: Redução drástica no tamanho do bundle inicial e carregamento mais rápido

# Lazy Loading e Suspense

React.lazy

- Carrega componente apenas quando necessário
- Reduz bundle inicial

```
const Dashboard = React.lazy(() => import('./Dashboard'));
```

# Lazy Loading e Suspense

Suspense

- Exibe fallback enquanto carrega
- Pode ser usado com lazy loading

```
<Suspense fallback={<Loading />}>
   <Dashboard />
</Suspense>
```

# Lazy Loading e Suspense

```
const HeavyComponent = React.lazy(() =>
  import('./HeavyComponent')
);
function App() {
  return (
    <Suspense fallback={<Spinner />}>
      <HeavyComponent />
    </Suspense>
  );
}
```

# Lazy Loading e Suspense

- Limitações do Suspense
  - Uso principal para code splitting
  - Data fetching depende da abordagem adotada

```
<Suspense fallback={<Loading />}>
   <Dashboard />
</Suspense>
```

# Análise de Performance

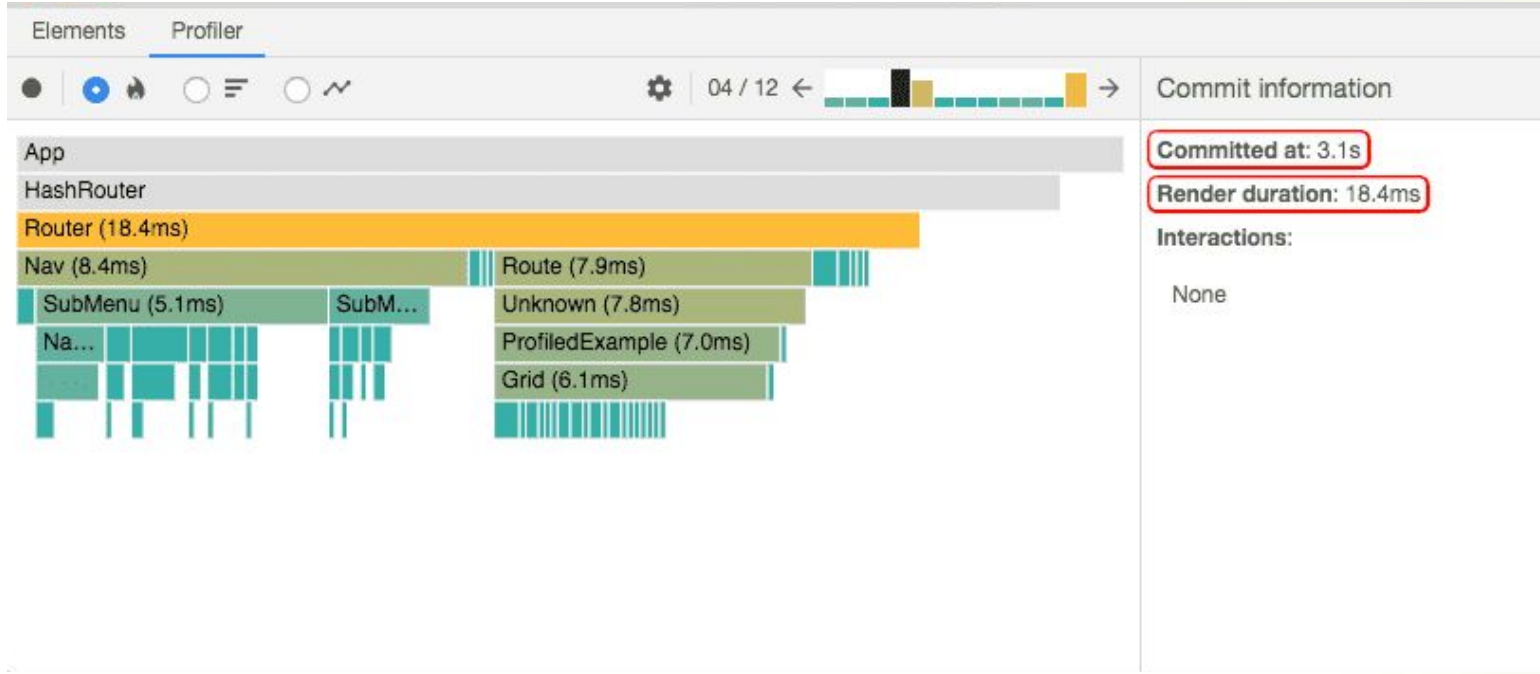
FERRAMENTAS

- React DevTools (Profiler)
- Chrome DevTools

# Análise de Performance

React Profiler

- Permite:
  - Medir tempo de render
  - Identificar componentes lentos
  - Comparar renders



# Análise de Performance

Exemplo de Análise

- Perguntas-chave:
  - Qual componente renderiza mais?
  - Renderiza sem necessidade?
  - Quanto tempo consome?

# React Profiler

COMO INSTALAR E INICIAR

1. Instale o React Developer Tools

- No navegador: React Developer Tools (extensão oficial)
- Sem essa extensão, o Profiler não aparece

2. Abra a aplicação React

- Rode normalmente (npm start para CRA, npm run dev para o Vite)
- Abra a aplicação no navegador

# React Profiler

COMO INSTALAR E INICIAR

3. Abra o DevTools do navegador

- ➢ F12 ou Ctrl + Shift + i
- ➢ Ou botão direito → Inspecionar

4. Vá até a aba Profiler

# React Profiler

COMO UTILIZAR

- Iniciar gravação
  - Clique no botão Record (●)
- Interaja com a aplicação
  - Clique em botões
  - Digite em inputs
  - Navegue entre páginas
- Parar gravação
  - Clique em Stop (■)
- O React vai mostrar:
  - Quais componentes renderizaram
  - Quantas vezes renderizaram
  - Quanto tempo cada render levou

# Boas Práticas

- Nunca otimizar por intuição
  - Usar o React Profiler para identificar gargalos
- Manter o estado próximo de onde é usado
  - Evitar elevar o estado desnecessariamente

# Boas Práticas

- Mudanças no Context API re-renderizam consumidores
  - Dividir contextos grandes em menores
- Ficar atento às novas ferramentas
  - O React Compiler ajuda com memoização automática

# Exercícios

- Ler e praticar a seção de useCallback do W3Schools

https://www.w3schools.com/react/react_usecallback.asp

- Ler e praticar a seção de useMemo do W3Schools

https://www.w3schools.com/react/react_usememo.asp

# Exercício: Gerenciador de Tarefas

- Ao longo deste curso, vamos desenvolver progressivamente uma aplicação React completa, aplicando os conceitos estudados em cada aula
- O projeto será construído de forma incremental: A cada novo conteúdo, novas funcionalidades deverão ser adicionadas
- Ao final do curso, você teremos:
  - Uma aplicação React funcional
  - Um projeto organizado, testado e otimizado
  - Um deploy público em ambiente de produção

# Exercício: Gerenciador de Tarefas

- O sistema consiste em uma aplicação web SPA (Single Page Application) para gerenciamento de projetos e tarefas, permitindo que o usuário:
  - Visualize projetos cadastrados
  - Acesse tarefas associadas a cada projeto
  - Crie, edite, conclua e remova tarefas
  - Navegue entre diferentes páginas da aplicação
  - Tenha seus dados persistidos localmente

# Exercício: Gerenciador de Tarefas

- A aplicação deverá conter, no mínimo, os seguintes componentes:
  - App
  - Header
  - ProjectList
  - TaskList
  - TaskItem
  - TaskForm

# Exercício: Gerenciador de Tarefas

- **Entregas da aula de hoje: Performance e Otimização**
  - Usar o React DevTools (Profiler) para ver que todos os itens da lista renderizam quando um só muda
  - Aplicar React.memo no componente TaskItem
  - Aplicar useCallback na função de deletar tarefa
- Funcionalidade: Otimizar a lista para não re-renderizar tarefas que não mudaram

# Exercício: Gerenciador de Tarefas

- **Entregas da aula de hoje: Performance e Otimização**
- Conceitos implementados:
  - Uso de React.memo, useCallback
  - Lazy loading de componentes ou rotas
  - Análise de re-renderizações
- Resultado Visual:
  - O "flash" do DevTools diminui, indicando que apenas o item alterado foi renderizado